Learning Efficient Prompt Selection for Large Language Models

Artificial Intelligence Problem Statement

Hardly Human — IIT Dharwad

Mentor: Prof. B. N. Bharath

1. Problem Statement

Large Language Models (LLMs) often rely on carefully designed prompts, instructions, or examples to produce accurate responses. However, including many prompts or examples increases token usage and computational cost. In practical systems, it is desirable to select a small subset of prompts that maximizes response quality while minimizing token usage.

This project aims to design a system that learns to select the most useful prompts for a given user query. The problem will be modeled as an online learning or contextual bandit problem, where the system observes a query, selects a subset of candidate prompts, and receives feedback based on the quality of the generated response.

Over time, the system should learn which prompts work best for different types of queries while respecting a token budget constraint.

Students will build a prototype that integrates an LLM with an adaptive prompt selection module and evaluate how prompt selection impacts response quality, token usage, and latency.

2. Provided Resources & Required Setup

Teams will receive the following resources:

- Access to a base Large Language Model through an API or open-source model.
- A predefined pool of candidate prompts and prompt templates.
- Example user queries for development and testing.

Recommended development setup:

Component	Requirement
Programming Language	Python
Frameworks	PyTorch / TensorFlow (optional)
Libraries	LLM APIs, bandit libraries, prompt frameworks
Evaluation Tools	Token usage, latency, response quality metrics

Training prompts or datasets provided during development will not be used during final evaluation.

3. Tasks to be Implemented

Teams must design and implement a system that adaptively selects prompts for an LLM-based application.

3.1 Prompt Pool Construction

Create a set of candidate prompts that assist the LLM in answering queries for a doctor appointment application. Prompts may include:

- Instruction prompts
- Example-based prompts
- Domain-specific contextual prompts

3.2 Prompt Selection Module

Develop an algorithm that selects a subset of prompts for each incoming query while respecting a token budget constraint.
Possible strategies include:

- Contextual bandits
- Online learning algorithms
- Heuristic prompt selection methods

3.3 LLM Integration

The selected prompts should be combined with the user query and passed to the LLM to generate responses.

3.4 Feedback and Learning

The system should receive feedback based on:

- Quality of the generated response
- Token usage
- Latency

This feedback should be used to improve prompt selection over time.

3.5 Application Scenario

The prototype should demonstrate the system within a doctor appointment assistant application where queries may include:

- Booking appointments
- Checking doctor availability
- Requesting information about medical services

4. Expected Outcomes

Teams are expected to implement:

- An online learning algorithm for selecting **appropriate prompts**
- A prompt selection framework for LLM-based systems
- Exploration of learning strategies such as contextual bandits or heuristic methods
- Experiments analyzing trade-offs between prompt length, token cost, and response quality
- A working prototype demonstrating adaptive prompt selection

5. Eligibility Requirements

Teams applying for this project should satisfy the following:

- Teams may consist of **1–4 students**.
- At least **one member** should have a **Computer Science or Mathematics and Computing background**.
- Teams should have a **good understanding of calculus**.
- Good programming skills are required (at least one language: **C, C++, or Python**).
- If at least one team member has completed a project related to **Machine Learning**, it will be an added advantage.
- Most importantly, students should be **motivated and hardworking**.

6. Selection Process

If the number of applying teams exceeds the available slots, teams will be **shortlisted based on a small evaluation conducted by Prof. B. N. Bharath**.

7. Final Submission Requirements

Each team must submit the following.

7.1 Source Code

Complete implementation of the prompt selection system including the learning module and LLM integration.

7.2 Demonstration Video (Maximum 3 minutes)

The video should demonstrate:

- Query input to the system
- Prompt selection process
- Response generation using the LLM
- Adaptation of prompt choices over time

7.3 Technical Document (Maximum 6 pages)

The report must include:

- System architecture
- Prompt selection strategy
- Learning algorithm used
- Experimental setup and evaluation
- Analysis of results

8. Evaluation

Submissions will be evaluated based on:

- Prompt selection effectiveness
- Response quality
- Token efficiency
- System design and documentation
- Demonstration of adaptive learning

Additional dates and resources will be announced later.